In [1]:
```
# Activity 4.01
```

In [2]:
```
# pandas for loading the car data
import pandas as pd
```

In [3]:
```
# link to the car dataset
file_url = 'https://raw.githubusercontent.com/PacktWorkshops/The-Applied-Artificial-Intelligence-Workshop/master/Datasets/car.csv'
```

In [4]:
```
# reading the csv into a dataframe
df = pd.read_csv(file_url)
```

In [5]:
```
# quick look at the first rows, columns are all text like "vhigh", "low" etc
df.head()
```

Out[5]:

	buying	maintenance	doors	persons	luggage_boot	safety	class
0	vhigh	vhigh	2	2	small	low	unacc
1	vhigh	vhigh	2	2	small	med	unacc
2	vhigh	vhigh	2	2	small	high	unacc
3	vhigh	vhigh	2	2	med	low	unacc
4	vhigh	vhigh	2	2	med	med	unacc

In [6]:
```
# sklearn's preprocessing has the label encoder i need
from sklearn import preprocessing
```

In [7]:
```
# helper that turns one text column into numbers, same category always gets same number
def encode(data_frame, column):
    label_encoder = preprocessing.LabelEncoder()
    label_encoder.fit(data_frame[column].unique())
    return label_encoder.transform(data_frame[column])
```

In [8]:
```
# running the encoder on every column in the dataframe
for column in df.columns:
    df[column] = encode(df, column)
```

In [9]:
```
# checking it worked, everything should be numbers now
df.head()
```

Out[9]:

	buying	maintenance	doors	persons	luggage_boot	safety	class
0	3	3	0	0	2	1	2
1	3	3	0	0	2	2	2
2	3	3	0	0	2	0	2
3	3	3	0	0	1	1	2
4	3	3	0	0	1	2	2

In [10]:
```
# pulling the class column out of df, it becomes the label and df keeps just the features
label = df.pop('class')
```

In [11]:
```
# model_selection has the train/test split function
from sklearn import model_selection
```

In [12]:
```
# 90/10 split, random_state=88 so results are repeatable
features_train, features_test, label_train, label_test = model_selection.train_test_split(df, label, test_size=0.1, random_state=88)
```

In [13]:
```
# importing the decision tree classifier
from sklearn.tree import DecisionTreeClassifier
```

In [14]:
```
# creating the model with default settings
decision_tree = DecisionTreeClassifier()
```

In [15]:
```
# training it on the training set
decision_tree.fit(features_train, label_train)
```

Out[15]:

▾ DecisionTreeClassifier

▸ Parameters

▸ Fitted attributes

In [16]:
```
# accuracy on the test set, data the model hasn't seen
decision_tree.score(features_test, label_test)
```

Out[16]:
```
0.953757225433526
```

In [17]:
```
# classification report gives precision, recall and f1 for each class
from sklearn.metrics import classification_report
```

In [18]:
```
# printing the report, accuracy alone can hide weak classes so checking per class
print(classification_report(label_test, decision_tree.predict(features_test)))
```

```
                precision    recall  f1-score   support

           0       0.87      0.98      0.92        42
           1       0.89      0.89      0.89         9
           2       0.99      0.96      0.98       114
           3       1.00      0.75      0.86         8

    accuracy                           0.95       173
   macro avg       0.94      0.89      0.91       173
weighted avg       0.96      0.95      0.95       173
```

In [19]:
```
# Activity 4.02
```

In [20]:
```
# random forest classifier, it's an ensemble of decision trees
from sklearn.ensemble import RandomForestClassifier
```

In [21]:
```
# 100 trees, max depth 6, random_state so the results are repeatable
random_forest_classifier = RandomForestClassifier(n_estimators=100, max_depth=6, random_state=168)
```

In [22]:
```
# training on the same train set as the decision tree
random_forest_classifier.fit(features_train, label_train)
```

Out[22]:

▾ RandomForestClassifier ⓘ ?

▸ Parameters

▸ Fitted attributes

In [23]:
```
# predicting classes for the test set
rf_preds_test = random_forest_classifier.predict(features_test)
rf_preds_test
```

Out[23]:
```
array([0, 0, 2, 0, 0, 2, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       0, 2, 2, 3, 2, 2, 2, 2, 0, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 0, 2, 0, 0, 0, 2, 2, 0, 2, 0, 2, 0, 2, 2, 2, 0, 2, 2, 2, 2, 0,
       0, 2, 2, 2, 0, 2, 3, 2, 2, 2, 0, 2, 2, 2, 2, 2, 0, 0, 0, 2, 2, 2,
       2, 2, 2, 2, 0, 2, 0, 0, 2, 2, 0, 2, 2, 2, 0, 0, 2, 0, 2, 2, 2, 2,
       0, 2, 2, 0, 2, 2, 3, 0, 2, 2, 2, 2, 0, 0, 2, 2, 2, 0, 2, 2, 2, 0,
       2, 2, 2, 2, 2, 2, 2, 0, 3, 3, 2, 0, 0, 2, 2, 2, 0, 0, 2, 2, 0, 2,
       2, 2, 2, 2, 0, 0, 2, 2, 2, 0, 2, 2, 2, 2, 0, 2, 0, 0, 2])
```

In [24]:
```
# zero_division=0 tells sklearn to report 0 for classes with no predictions, silences the warning
print(classification_report(label_test, rf_preds_test, zero_division=0))
```

	precision	recall	f1-score	support
0	0.67	0.76	0.71	42
1	0.00	0.00	0.00	9
2	0.92	0.96	0.94	114
3	0.83	0.62	0.71	8
accuracy			0.84	173
macro avg	0.60	0.59	0.59	173
weighted avg	0.80	0.84	0.82	173

In [25]:
```
# confusion matrix needs its own import
from sklearn.metrics import confusion_matrix
```

In [26]:
```
# rows are actual classes, columns are predicted, shows exactly which classes get mixed up
confusion_matrix(label_test, rf_preds_test)
```

Out[26]:
```
array([[ 32,   0,  10,   0],
       [  8,   0,   0,   1],
       [  5,   0, 109,   0],
       [  3,   0,   0,   5]])
```

In [27]:
```
# how much each feature mattered to the forest
rf_varimp = random_forest_classifier.feature_importances_
rf_varimp
```

Out[27]:
```
array([0.12676384, 0.10366314, 0.02119621, 0.35266673, 0.05915769,
       0.33655239])
```

In [28]:
```
# extra trees is similar to random forest but splits are more random
from sklearn.ensemble import ExtraTreesClassifier
```

In [29]:
```
# same settings as the forest so the comparison is fair
extra_trees_classifier = ExtraTreesClassifier(n_estimators=100, max_depth=6, random_state=168)
```

In [30]:
```
# training extra trees
extra_trees_classifier.fit(features_train, label_train)
```

Out[30]:

▾ ExtraTreesClassifier

▸ Parameters

▸ Fitted attributes

In [31]:
```
# test set predictions from extra trees
et_preds_test = extra_trees_classifier.predict(features_test)
et_preds_test
```

Out[31]:
```
array([0, 0, 2, 0, 2, 2, 3, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       0, 2, 2, 0, 2, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 0, 2, 0, 0, 0, 2, 2, 2, 2, 0, 2, 0, 2, 2, 2, 0, 2, 2, 2, 2, 0,
       0, 2, 2, 2, 2, 2, 0, 2, 2, 2, 0, 2, 2, 2, 2, 2, 0, 0, 0, 2, 2, 2,
       2, 2, 2, 2, 0, 2, 0, 0, 2, 2, 0, 2, 2, 2, 2, 0, 2, 0, 2, 2, 2, 2,
       0, 2, 2, 0, 2, 2, 0, 0, 2, 2, 2, 2, 0, 2, 2, 2, 2, 0, 2, 2, 2, 0,
       2, 2, 2, 2, 2, 2, 2, 0, 0, 0, 2, 0, 0, 2, 2, 2, 0, 0, 2, 2, 0, 2,
       2, 2, 2, 2, 0, 0, 2, 2, 2, 0, 2, 2, 2, 2, 2, 2, 0, 0, 2])
```

In [32]:
```
# same fix as the forest, report 0 for classes with no predictions instead of warning
print(classification_report(label_test, et_preds_test, zero_division=0))
```

```
              precision    recall  f1-score   support

           0       0.61      0.67      0.64        42
           1       0.00      0.00      0.00         9
           2       0.89      0.98      0.93       114
           3       1.00      0.12      0.22         8

    accuracy                           0.82       173
   macro avg       0.62      0.44      0.45       173
weighted avg       0.78      0.82      0.78       173
```

In [33]:
```
# confusion matrix for extra trees
confusion_matrix(label_test, et_preds_test)
```

Out[33]:
```
array([[ 28,   0,  14,   0],
       [  9,   0,   0,   0],
       [  2,   0, 112,   0],
       [  7,   0,   0,   1]])
```

In [34]:
```
# feature importances for extra trees, checking if it ranks features like the forest did
et_varimp = extra_trees_classifier.feature_importances_
et_varimp
```

Out[34]:
```
array([0.08844544, 0.0702334 , 0.01440408, 0.37662014, 0.05965896,
       0.39063797])
```

In [ ]: